

Digital Clock -- Mentoring Guide

A Streamlit project with three small, self-contained pieces: a live ticking clock, a server-side timezone lookup, and a stopwatch. This guide walks through how each one works and why it's built the way it is, then closes with exercises to extend it yourself.

1. Why a clock can't just be `st.write(datetime.now())`

Streamlit's whole script re-runs top to bottom whenever something happens -- a click, a text edit, a call to `st.rerun()`. Nothing makes it re-run once a second on its own. If you wrote `st.write(datetime.now())` at the top of this app, it would print the time the page was *loaded*, and then sit there frozen until the next interaction -- not what a clock needs.

That's the problem this app's first section solves.

2. The live clock: `st.iframe`

`st.iframe()` embeds raw HTML (and any JavaScript inside it) directly into the page, in a sandboxed iframe. Unlike the rest of a Streamlit app, that JavaScript keeps running in the browser completely independently of Streamlit's rerun model -- it never talks to the Python server again after the page loads.

```
<script>
  function tick() {
    const now = new Date();
    const pad = (n) => String(n).padStart(2, "0");
    document.getElementById("clock").textContent =
      pad(now.getHours()) + ":" + pad(now.getMinutes()) + ":" + pad(now.getSeconds());
  }
  tick();
  setInterval(tick, 1000);
</script>
```

`setInterval(tick, 1000)` is plain JavaScript: call `tick()` every 1000ms. This is the only part of the whole app that updates without a Streamlit rerun -- worth noticing as the general pattern for "anything that must animate or tick on its own" in a Streamlit app: reach for HTML/JS via `st.iframe`, not Python-side polling loops.

3. Server-side time: the `zoneinfo` standard-library module

The second section is ordinary Python, computed fresh on every rerun. `zoneinfo` (standard library since Python 3.9, no `pip install` needed) gives `datetime` real timezone awareness:

```
from zoneinfo import ZoneInfo
from datetime import datetime

now_in_tz = datetime.now(ZoneInfo("Asia/Kolkata"))
```

Because this part has no JavaScript ticking it, it's only ever as fresh as the last rerun -- which is exactly why there's an explicit "Refresh" button next to it. This contrast (JS clock: always live; Python clock: live only when Streamlit reruns) is the single most important thing this app demonstrates about how Streamlit works.

4. The stopwatch: `session_state`, not a running counter

A natural first instinct for a stopwatch is a loop that increments a counter once a second. That doesn't work in Streamlit -- a script that never returns blocks the whole app from handling any other interaction. Instead, this app stores *when* the stopwatch was started, and computes elapsed time from that timestamp on every rerun:

```
if st.button("■ Start", disabled=st.session_state.stopwatch_start is not None):
    st.session_state.stopwatch_start = time.time()
    st.rerun()
```

Stopping adds the just-elapsed interval (`time.time() - stopwatch_start`) onto a running total, `stopwatch_elapsed`, and clears `stopwatch_start` back to `None`. While running, the displayed elapsed time is computed as `stopwatch_elapsed + (time.time() - stopwatch_start)` -- the sum of all previously-completed stop/start segments, plus whatever has ticked by since the current segment began.

Every button here calls `st.rerun()` right after mutating state, for the same reason the to-do list and expense tracker projects do: without it, a state change made by one click only becomes visible on screen at the *next* rerun, not immediately -- try removing a `st.rerun()` call and clicking Start once to see the one-click lag for yourself.

Each button is also individually disabled based on current state (Start disabled while running, Stop disabled while stopped) -- a small guard against double-starting or double-stopping the same stopwatch.

5. Exercises

- **12-hour format toggle.** Add a checkbox that switches the live JS clock between 24-hour and 12-hour (AM/PM) display.
- **Lap times.** Add a "Lap" button to the stopwatch that appends the current elapsed time to a list in `session_state`, and display the list of laps below it (same list-in-session-state pattern the to-do list and expense tracker use).
- **Countdown timer.** Build a third section: let the user enter a number of seconds, and count down instead of up -- reuse the start-timestamp trick, but compute *remaining* time instead of elapsed time.
- **More timezones.** Replace the fixed `TIMEZONES` list with a searchable selectbox over all of `zoneinfo.available_timezones()` (hundreds of entries) instead of the six hand-picked ones.
- **World clock grid.** Show four or five timezones at once, side by side in columns, instead of one at a time via a selectbox.

Written as a teaching companion to `app.py` -- read this guide with the code open side by side.